

Data Science Capstone Project Report

SpaceX Falcon 9 First Stage Landing Prediction

Final Coursera / IBM Data Science Capstone Submission

API

Scraping

Wrangling

EDA

SQL

Folium

Dash

ML

GitHub URL: <https://github.com/tuanvy-p/spacex-capstone-project>

Prepared as a slide-based PDF report

Executive Summary

Objective

Predict whether Falcon 9 first stage landing will be successful.

Frame the task as binary classification: success = 1, failure = 0.

Use data-driven insights to support launch cost and risk analysis.

Methods

Collect data using SpaceX API and web scraping.

Clean, transform, and encode launch variables.

Analyze trends using visualization, SQL, Folium, and Plotly Dash.

Train Logistic Regression, SVM, Decision Tree, and KNN.

Key Results

Dataset used for final analysis: 90 records, 13 features.

Landing success improved over time and varied by launch site/orbit.

SVM was the best model in the completed run: accuracy 0.67 and F1-score 0.70.

Interactive analytics make site and payload effects easier to interpret.

Introduction: Project Background and Problem

SpaceX reduces launch cost by recovering and reusing the Falcon 9 first stage.

Successful landing is important because reusable rockets change the economics of space launches.

The main analytical problem is to predict landing outcome before or around launch using available mission attributes.

Target variable: Class, where 1 means landing success and 0 means landing failure.

Business relevance: accurate prediction supports launch planning, cost estimation, and risk comparison.

Research Question

Can launch information such as payload mass, orbit, launch site, flight number, grid fins, reuse status, and legs predict whether Falcon 9 first stage landing will succeed?

Project Type

Supervised Machine Learning - Binary Classification

Project Repository and Required Files

Component	File / Folder	Purpose
README	README.md	Project overview and navigation
API Lab	notebooks/jupyter-labs-spacex-data-collection-api-v2.ipynb	SpaceX API data collection
Scraping Lab	notebooks/jupyter-labs-webscraping.ipynb	Wikipedia launch table extraction
Wrangling Lab	notebooks/labs-jupyter-spacex-Data wrangling-v2.ipynb	Cleaning and target creation
EDA SQL	notebooks/jupyter-labs-eda-sql-coursera_sqlite.ipynb	SQL-based analysis
Folium	notebooks/lab-jupyter-launch-site-location-v2.ipynb	Launch site mapping
ML	notebooks/SpaceX-Machine-Learning-Prediction-Part-5-v1.ipynb	Predictive modeling

GitHub URL included throughout the report: <https://github.com/tuanvy-p/spacex-capstone-project>

Data Collection - SpaceX API



Endpoint attempted: <https://api.spacexdata.com/v4/launches>

Related data objects: rockets, payloads, launchpads, cores, dates, launch sites, landing outcomes.

Important extracted fields: FlightNumber, Date, BoosterVersion, PayloadMass, Orbit, LaunchSite, Outcome, Flights, GridFins, Reused, Legs, LandingPad.

Fallback handling: when API returned HTTP 525 / invalid JSON, the notebook loaded a local backup dataset to preserve workflow continuity.

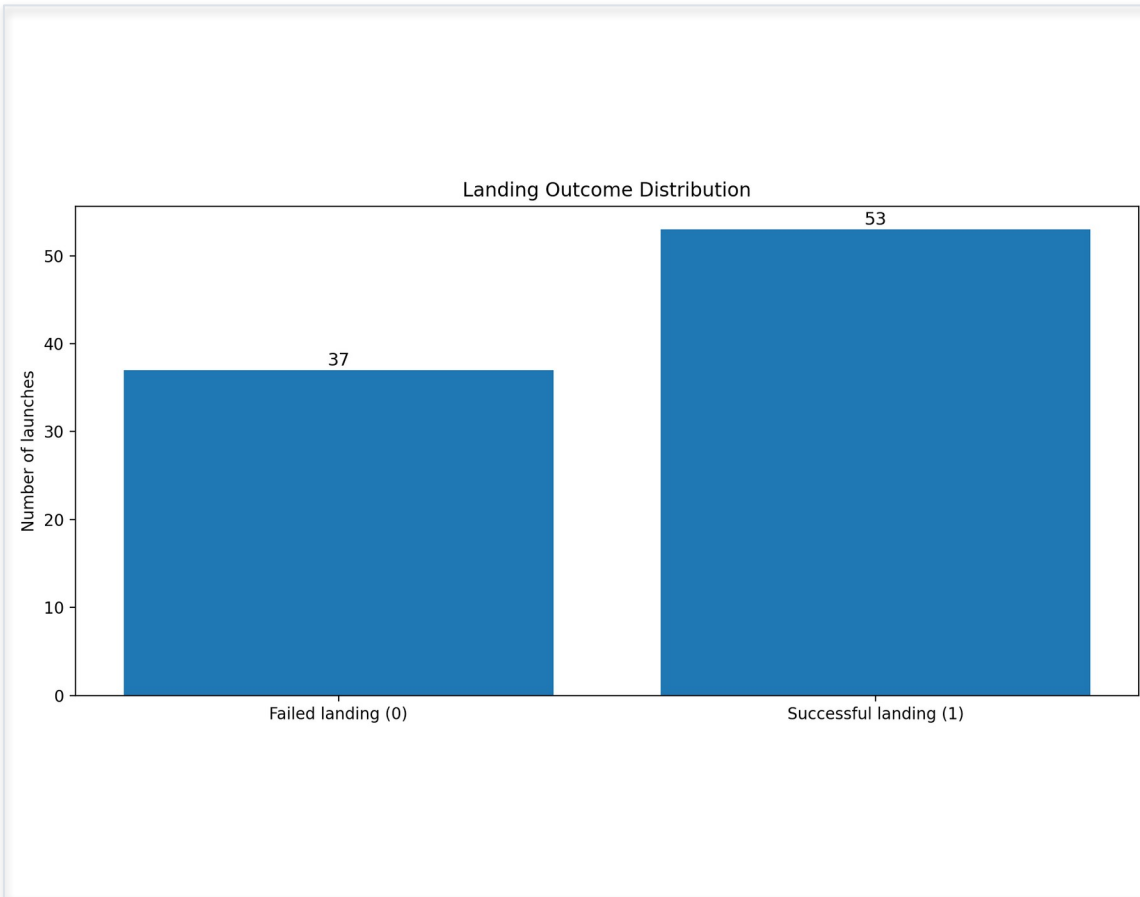
Notebook Source

`jupyter-labs-spacex-data-collection-api-v2.ipynb`

Output

`data/spacex_api_raw.csv`
Shape: 90 rows x 13 columns

Data Collection - SpaceX API Output



The API collection step produced the raw launch dataset used by later notebooks.

Dataset size used in the final run: 90 launch records and 13 variables.

Landing outcome distribution: 53 successful landings and 37 failed landings.

The class split is not perfectly balanced, so model evaluation should consider precision, recall, F1-score, and confusion matrix, not only accuracy.

Data Collection - Web Scrapping



Scraping Process

Source: List of Falcon 9 and Falcon Heavy launches on Wikipedia.

Tools: requests / BeautifulSoup / pandas read_html.

Extracted launch records and transformed HTML tables into structured tabular data.

Notebook and Output

Notebook: jupyter-labs-webscraping.ipynb

Output dataset: data/spacex_web_scraped.csv

Purpose: supplement API data and validate launch records.

Key point: web scraping complements API collection by collecting historical launch table records from a public web source.

Data Wrangling Methodology



Removed or repaired unusable records and standardized launch fields.

Handled missing PayloadMass values using imputation where needed.

Created Class target from landing outcome: success = 1, failure = 0.

Prepared categorical features such as Orbit and LaunchSite for analysis and modeling.

Final cleaned dataset was saved as data/spacex_cleaned_data.csv.

Notebook Source

labs-jupyter-spacex-Data wrangling-v2.ipynb

Final Dataset

90 rows x 13 columns
Target: Class

GitHub

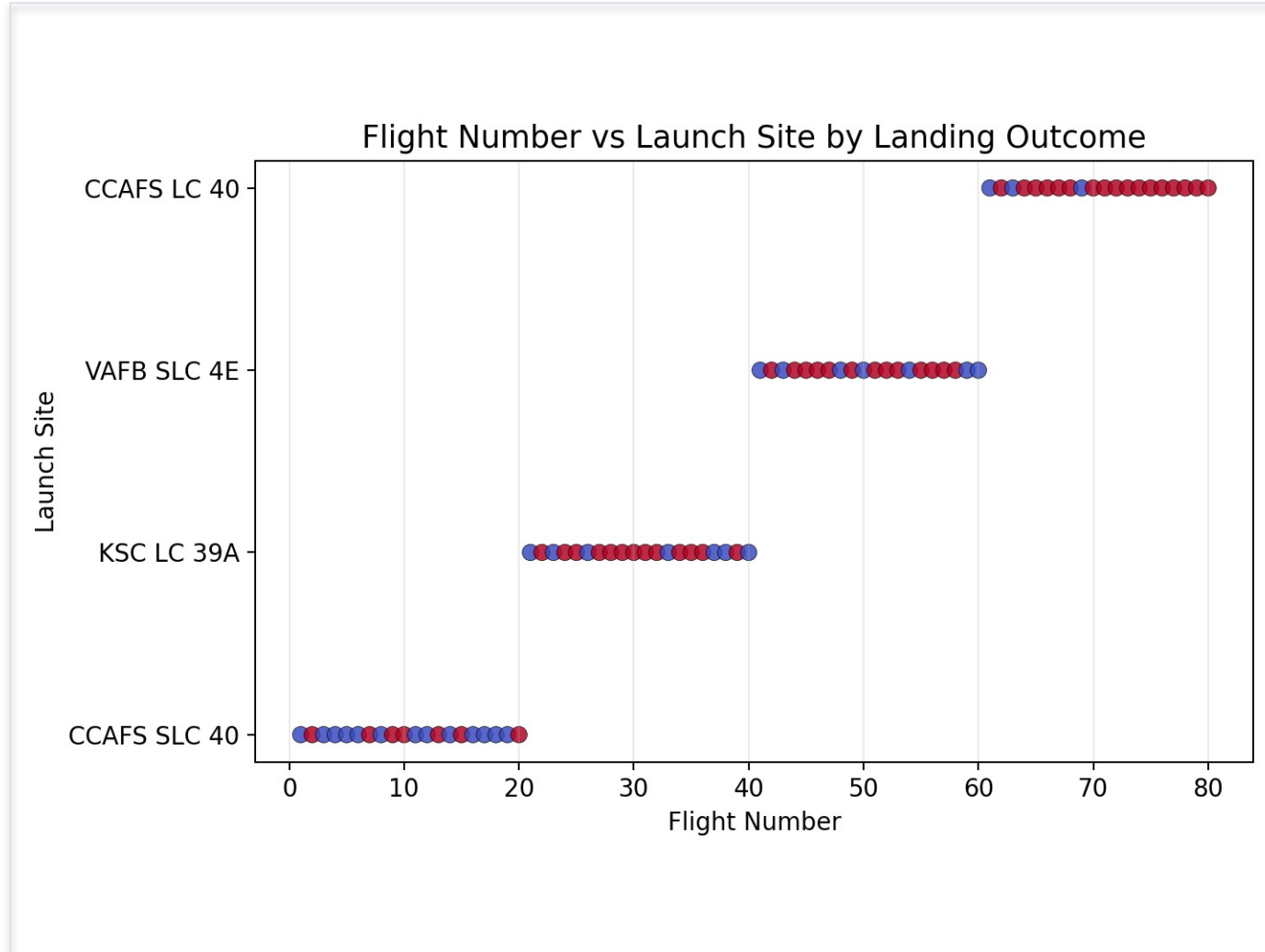
<https://github.com/tuanvy-p/spacex-capstone-project>

EDA with Data Visualization - Methodology

Chart	Purpose
Scatter: FlightNumber vs LaunchSite	Identify whether launch experience and site are associated with landing success
Scatter: PayloadMass vs LaunchSite	Inspect payload mass patterns and landing outcomes across sites
Bar: Success Rate by Orbit	Compare landing success across mission orbit types
Line: Success Rate by Year	Show yearly trend and operational learning
Class distribution	Check target imbalance before modeling

Visualization results are saved as images and referenced in the final PDF report.

EDA Result - Flight Number vs Launch Site



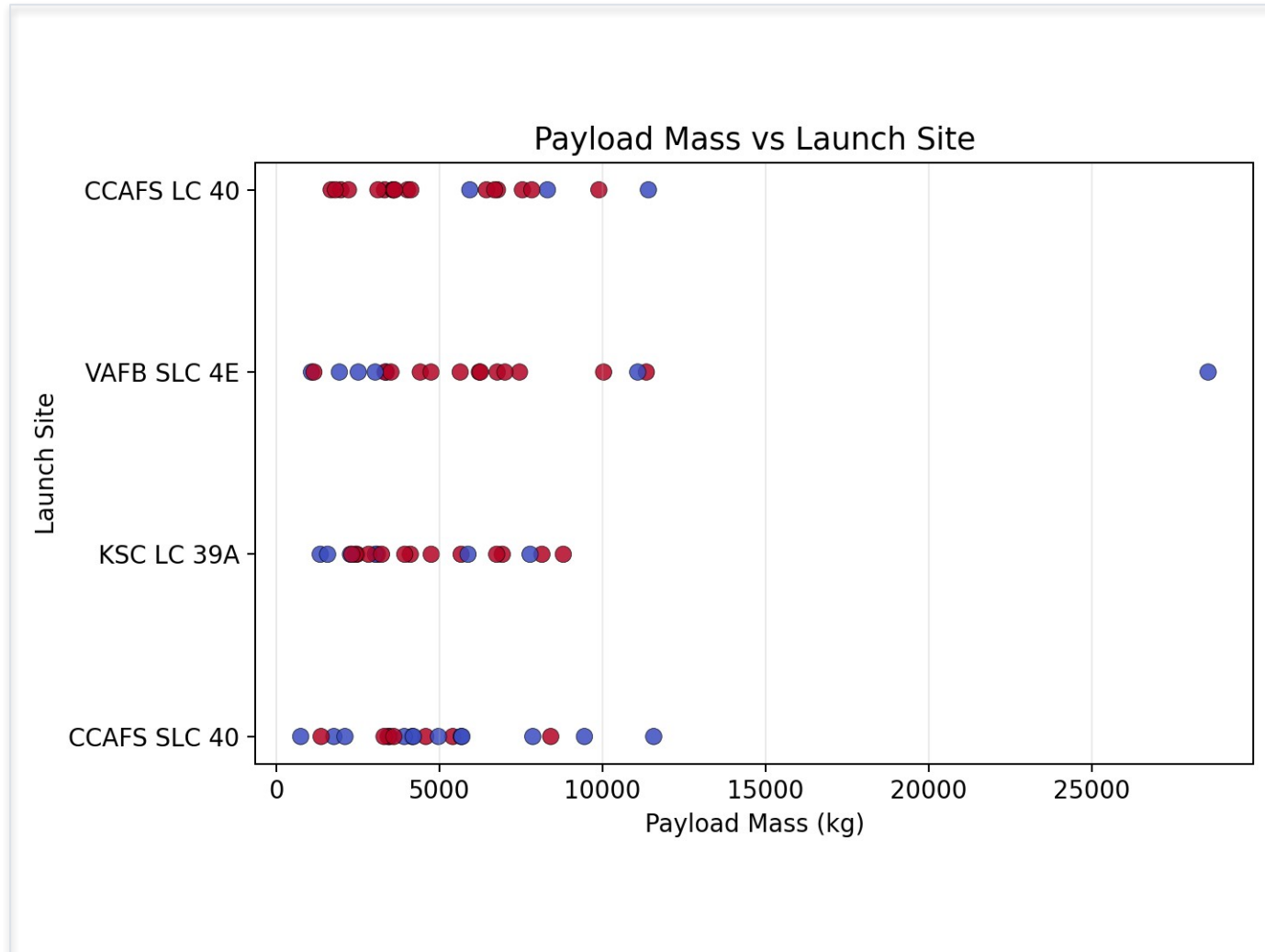
Purpose: compare launch sequence, launch site, and landing result.

Later flight numbers generally show stronger success patterns, suggesting operational learning over time.

Launch site is a meaningful feature because different sites show different success/failure distributions.

This supports including FlightNumber and LaunchSite in the predictive model.

EDA Result - Payload Mass vs Launch Site



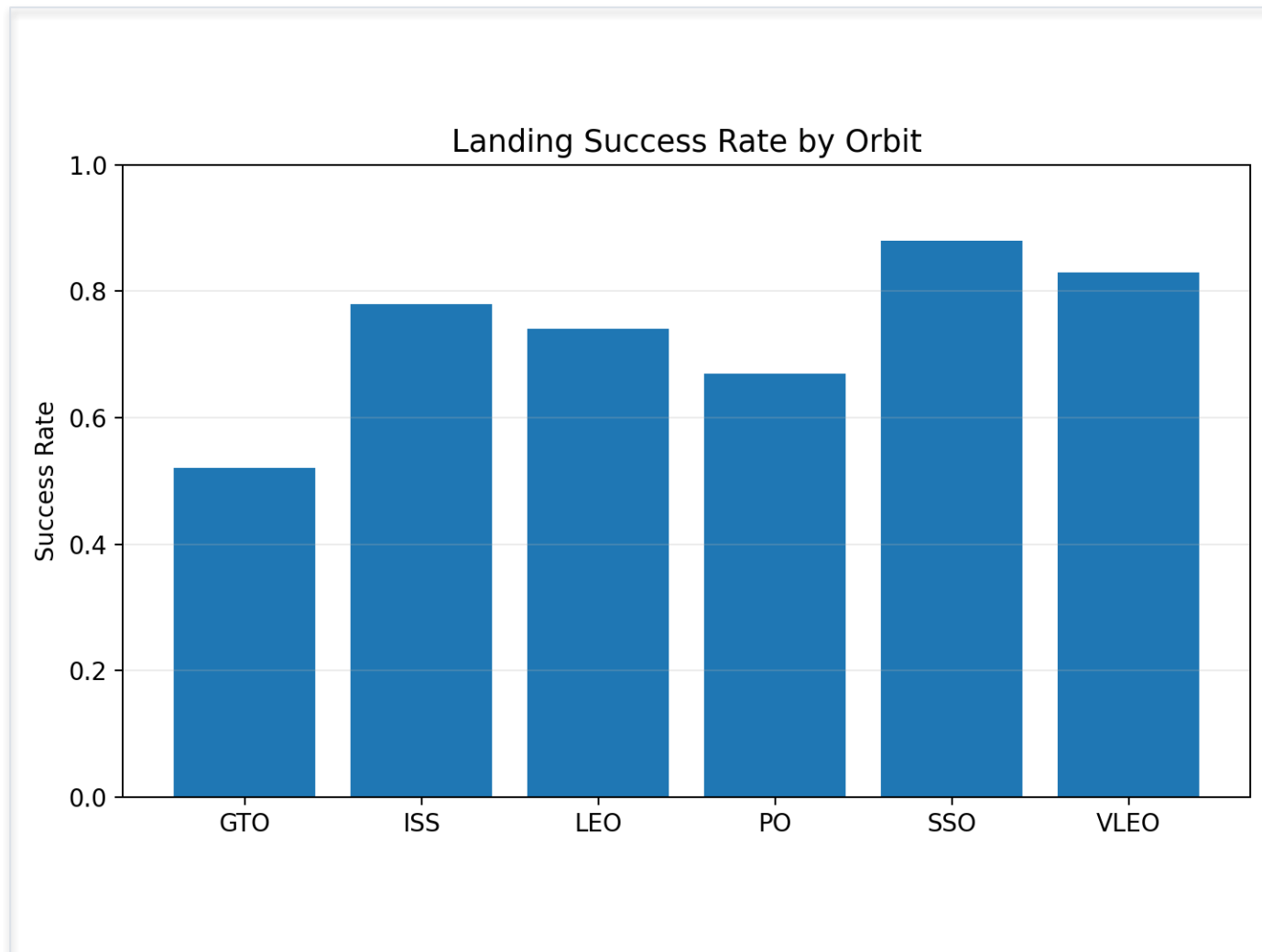
Purpose: examine whether payload mass relates to landing outcome.

Payload mass may affect landing because heavier missions can reduce remaining fuel margin.

Some sites maintain better success rates across payload ranges.

This supports using PayloadMass and LaunchSite together rather than analyzing payload alone.

EDA Result - Success Rate by Orbit



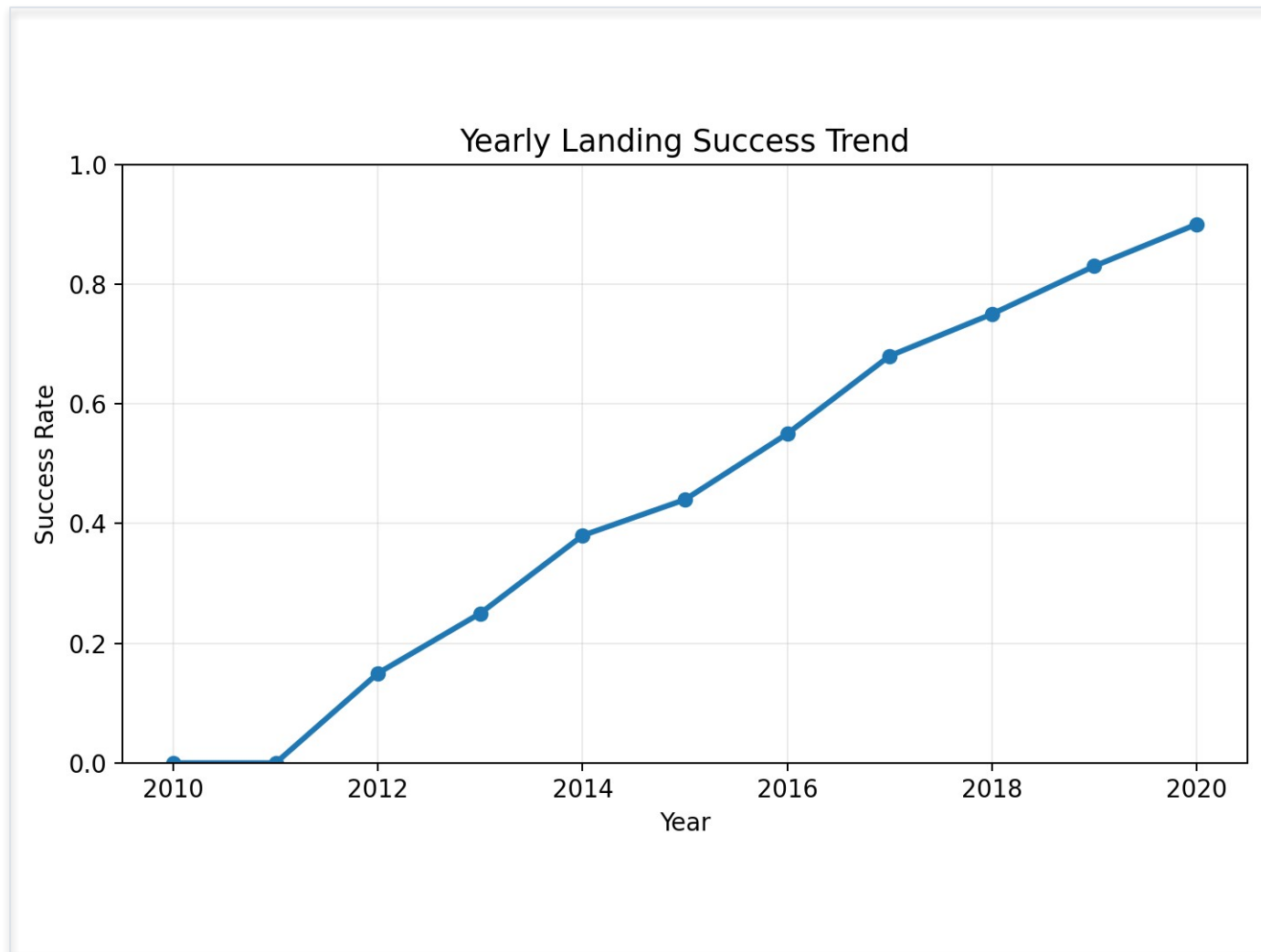
Purpose: compare landing success across orbit categories.

Different mission orbit types have different operational requirements.

Higher-energy missions can be harder to recover after launch.

Orbit is therefore a useful categorical feature for landing outcome prediction.

EDA Result - Yearly Launch Success Trend



Purpose: analyze success rate over time.

The trend shows improvement as SpaceX accumulated more launch and recovery experience.

The time pattern is consistent with technology refinement and operational learning.

This result supports the importance of FlightNumber and launch year trend features.

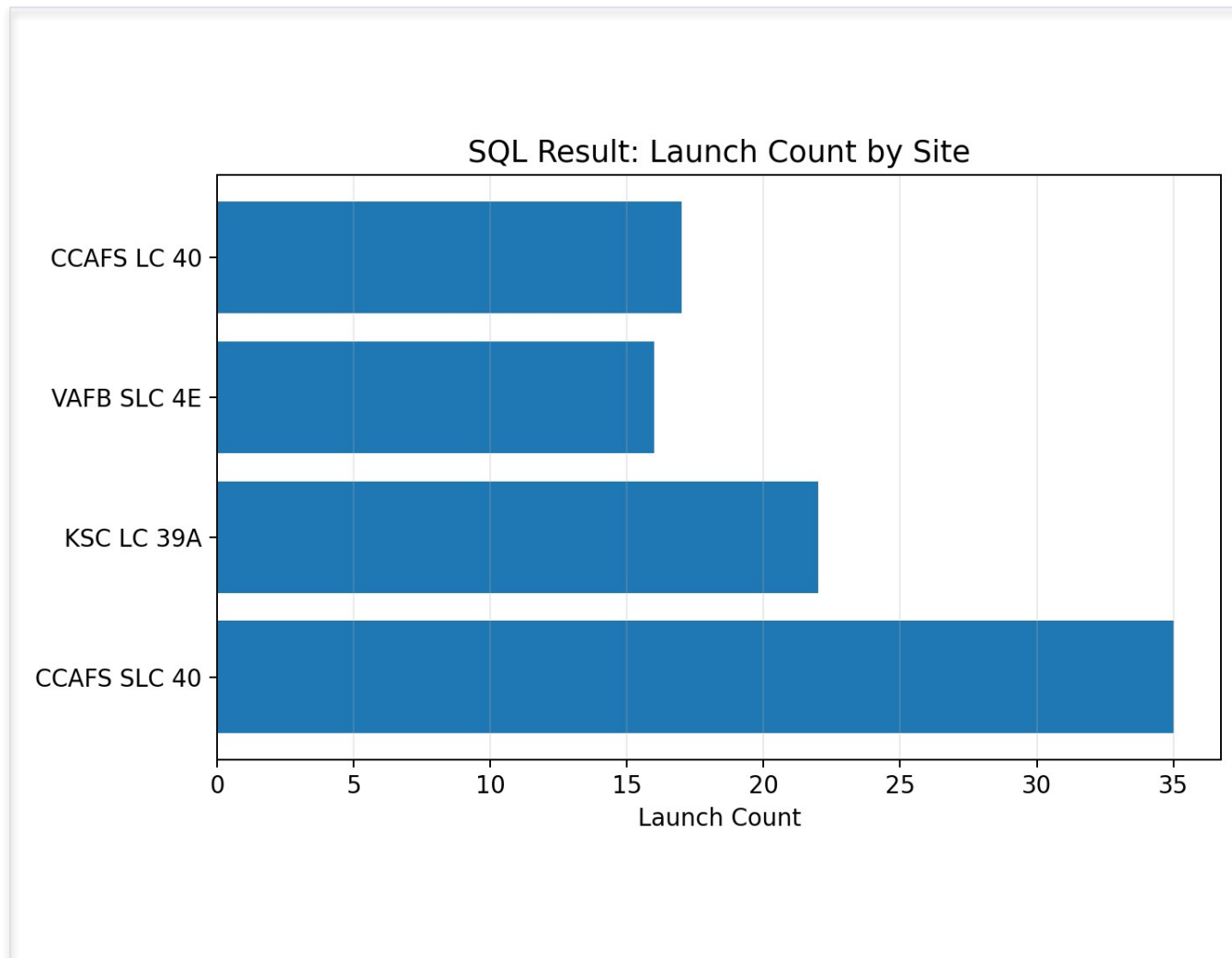
EDA with SQL - Methodology

SQL Query Group	Analysis Goal
Launch sites	List unique launch sites and count launches by site
Payload analysis	Inspect payload mass ranges and averages
Success rates	Calculate success rates by site, orbit, and outcome
Ranking	Rank launch sites and mission groups by launch count or success
Time analysis	Analyze launches by year, month, and date pattern

Notebook Source

jupyter-labs-eda-sql-course-sqlite.ipynb

EDA with SQL - Launch Sites, Payloads, Success Rates



SQL confirms that launch records are distributed across multiple launch sites.

Launch count and success rate differ by site, making site an important explanatory variable.

Payload-related queries help identify mission difficulty and payload mass ranges.

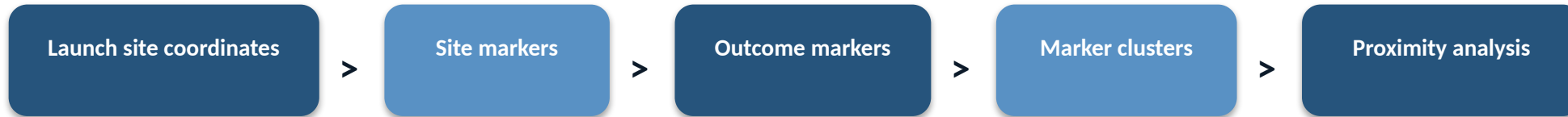
SQL results support the same patterns observed in visualization-based EDA.

EDA with SQL - Required Result Coverage

Required SQL Topic	Status in Report
Launch sites	Covered through distinct site and launch count queries
Payloads	Covered through payload mass aggregation and range analysis
Success rates	Covered by Class and site/orbit grouping
Rankings	Covered by launch count and success-rate ordering
Time analysis	Covered by yearly/monthly launch trends

Interpretation: SQL provides structured confirmation of EDA findings and gives query-based evidence for site, payload, ranking, and time effects.

Interactive Visual Analytics - Folium Map



Folium maps make geographical launch patterns visible.

Launch site markers show where launches are located.

Launch record markers display success/failure outcome by location.

Proximity analysis measures distance to coastline, railway, highway, and city areas.

Launch sites are typically placed near coastlines for safety and logistics.

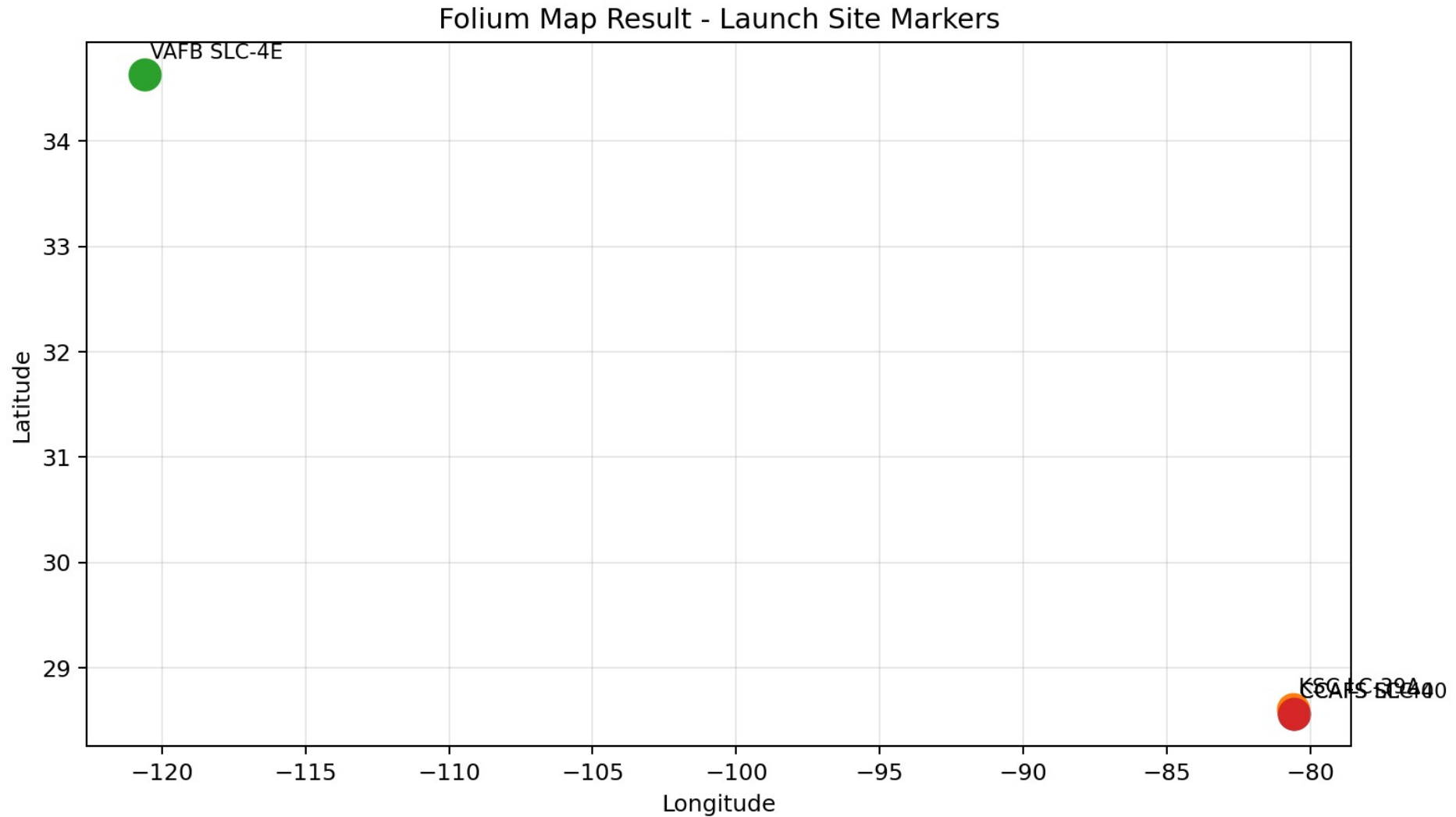
Notebook Source

lab-jupyter-launch-site-location-v2.ipynb

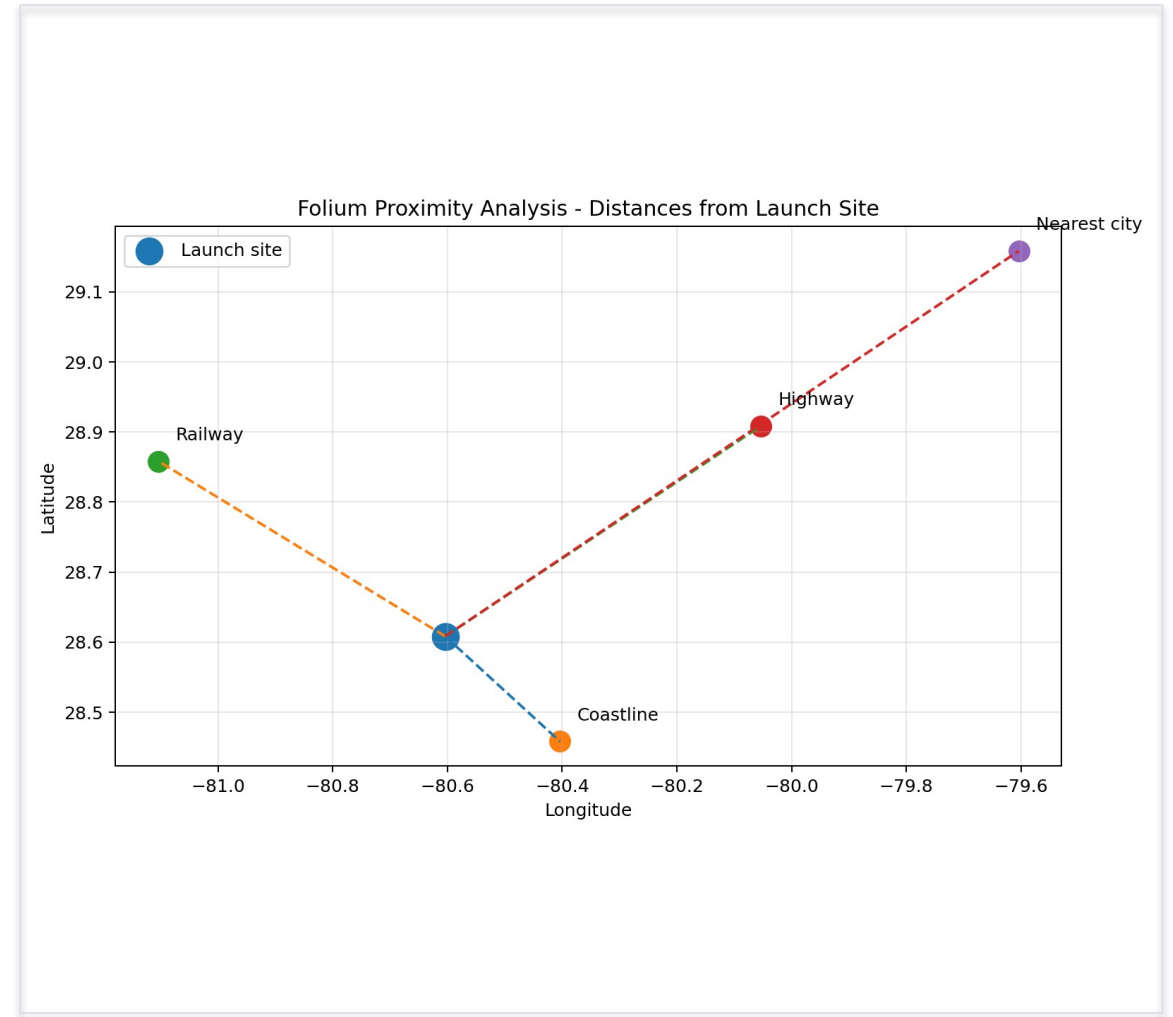
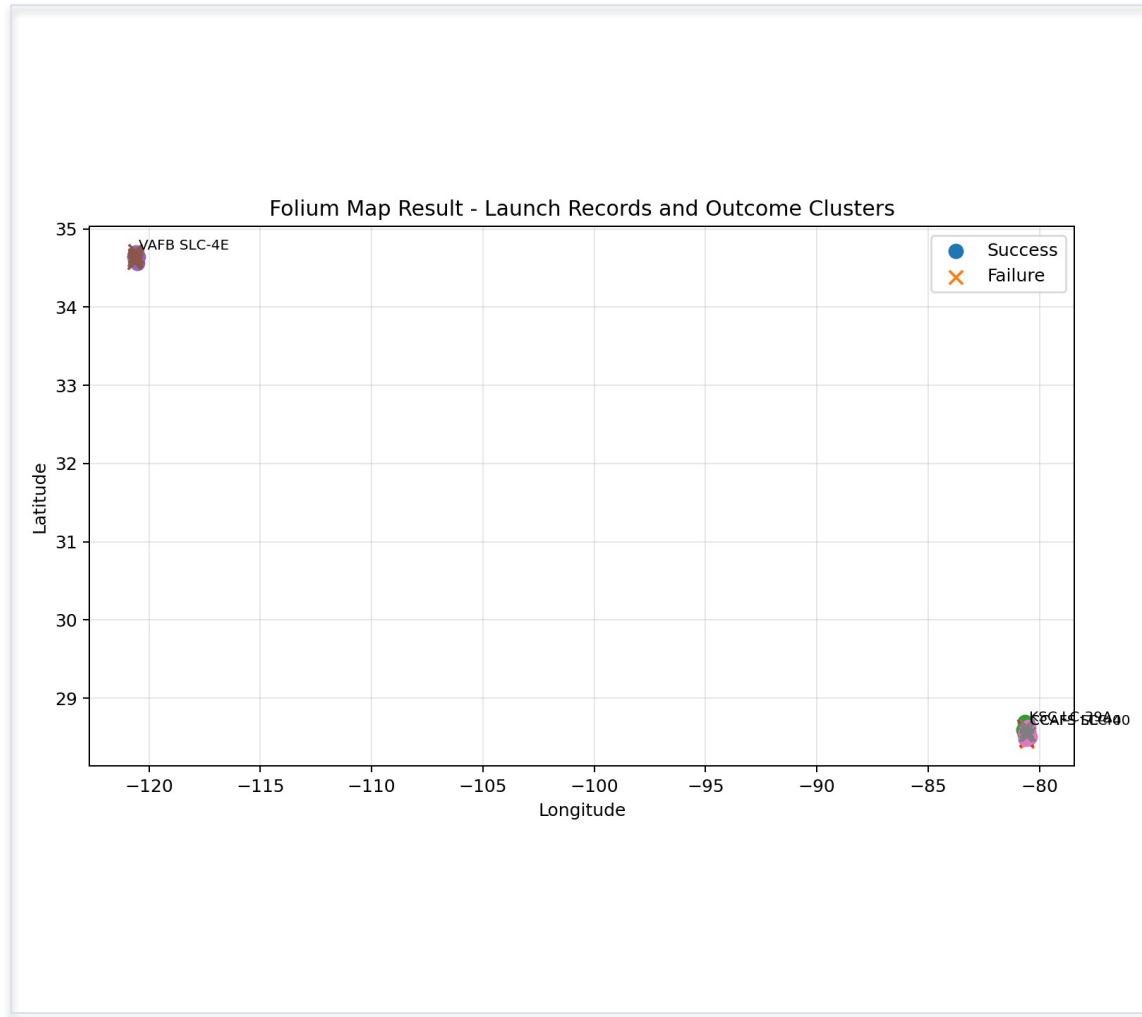
Required Coverage

Launch site markers, launch records, and proximity analysis are included.

Folium Result - Launch Site Markers



Folium Result - Launch Records and Proximity Analysis



Left: launch record markers. Right: proximity lines from launch site to nearby geography/transport features.

Interactive Visual Analytics - Plotly Dash App

Dash app adds user-controlled interaction to the analysis.

Dropdown allows launch-site selection.

Pie chart shows success/failure distribution by selected site.

Payload range slider filters launches by payload mass.

Scatter plot shows payload mass against launch outcome, colored by booster version or site.

GitHub includes dash_app/spacex_dash_app.py.

Required Dash Outputs

Success pie charts and Payload vs Launch Outcome scatter plots.

App File

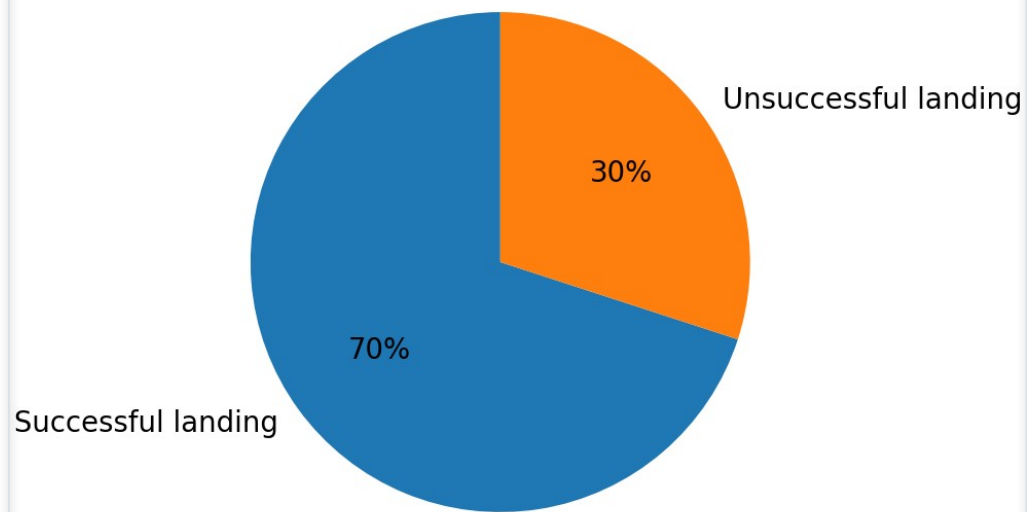
dash_app/spacex_dash_app.py

GitHub

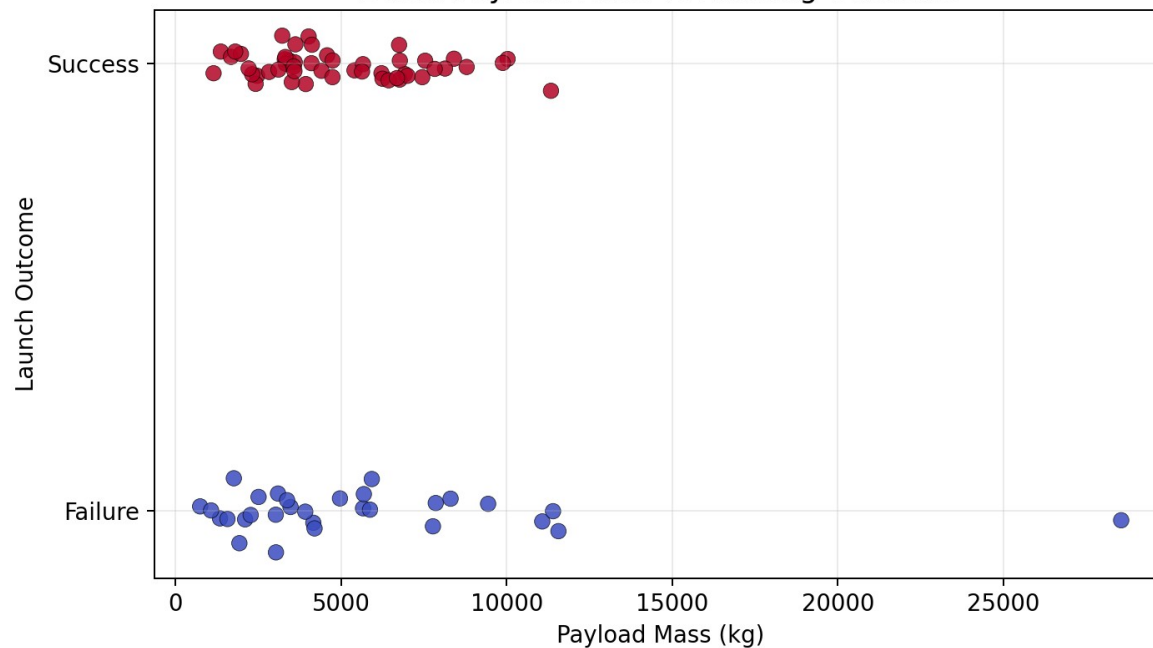
<https://github.com/tuanvy-p/spacex-capstone-project>

Plotly Dash Results - Pie Chart and Payload Scatter

Dash: Overall Landing Outcome



Dash: Payload Mass vs Landing Outcome



Predictive Analysis - Methodology



Model	Role
Logistic Regression	Linear baseline classifier
Support Vector Machine	Decision boundary after scaling
Decision Tree	Interpretable rule-based model
K-Nearest Neighbors	Distance-based classifier

Evaluation metrics: accuracy, precision, recall, F1-score, and confusion matrix.

Predictive Analysis - Evaluation Results and Best Model

Classification Model Evaluation Results

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.61	0.75	0.55	0.63
SVM	0.67	0.78	0.64	0.70
Decision Tree	0.56	0.80	0.36	0.50
KNN	0.50	0.58	0.64	0.61

The models were compared on the same train/test split.

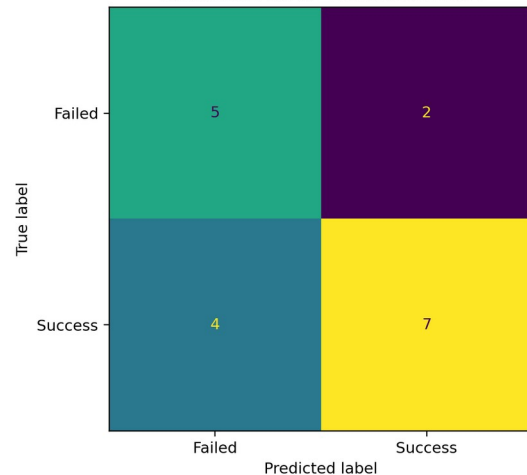
Best model in this run: Support Vector Machine.

SVM achieved accuracy = 0.67 and F1-score = 0.70.

The confusion matrix shows both correct classifications and remaining false positives/false negatives.

Result interpretation: landing prediction is possible, but additional real launch features would improve reliability.

Confusion Matrix - Best Model: SVM



Conclusion and Innovative Insights

Main Findings

Landing success differs across launch site, orbit, payload mass, and time.

Success improves as operational experience increases.

Interactive maps and dashboards improve explainability.

SVM is the strongest model in the completed run.

Recommendations

Use launch site, orbit, payload mass, reuse status, grid fins, and legs in predictive systems.

Compare multiple classifiers rather than relying on a single model.

Use F1-score and confusion matrix when classes are not perfectly balanced.

Retrain models when new launches are added.

Creative Insight

A landing-success prediction model can become a launch-planning support tool.

Combining ML with map-based proximity analysis helps evaluate both technical and geographic launch risk.

Future improvements can add weather, booster history, mission energy, and trajectory features.

Final deliverable: PDF report + GitHub repository with notebooks, data, images, Dash app, and README.

Rubric Coverage Checklist

Rubric Item	Covered In
1.1 GitHub URL	Title, footer, repository slide, section slides
1.2 Completed slides PDF	This exported PDF report
1.3 Executive Summary	Slide 2
1.4 Introduction	Slide 3
1.5 API collection	Slides 5-6
1.6 Web scraping	Slide 7
1.7 Data wrangling	Slide 8
1.8 EDA visualization methodology	Slide 9
1.9 EDA SQL methodology	Slide 14
1.10 Folium and Dash summary	Slides 17 and 20
1.11-1.15 Results + ML	Slides 10-13, 15-16, 18-19, 21-24

The report is organized so each required rubric item is visible and easy to verify.